Implement type checking.

typecheck.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef enum
{
 TYPE_INT,
 TYPE_FLOAT,
 TYPE_CHAR,
 TYPE_VOID,
 TYPE_ERROR
} DataType;
typedef struct TypeInfo
{
DataType type;
int isArray;
int arraySize;
} TypeInfo;
typedef struct Variable
{
char name[50];
TypeInfo typeInfo;
struct Variable* next;
} Variable;
Variable* symbolTable = NULL;
void addVariable(char* name, DataType type, int isArray, int size) {
Variable* var = (Variable*)malloc(sizeof(Variable));
strcpy(var->name, name);
var->typeInfo.type = type;
var->typeInfo.isArray = isArray;
var->typeInfo.arraySize = size;
var->next = symbolTable;
symbolTable = var;
char* typeStr = (type == TYPE_INT) ? "int" :
(type == TYPE_FLOAT) ? "float" :
(type == TYPE_CHAR) ? "char" : "void";
printf("Added variable %s: %s%s\n", name, typeStr, isArray ? " array" : "");
}
TypeInfo getVariableType(char* name) {
Variable* current = symbolTable;while (current) {
if (strcmp(current->name, name) == 0) {
return current->typeInfo;
}
current = current->next;
}
TypeInfo error = {TYPE_ERROR, 0, 0};
return error;
}
```

```
int areTypesCompatible(DataType type1, DataType type2) {
if (type1 == type2) return 1;
if ((type1 == TYPE_INT && type2 == TYPE_FLOAT) ||
(type1 == TYPE_FLOAT && type2 == TYPE_INT)) return 1;
return 0;
}
DataType checkBinaryOperation(char* var1, char* var2, char op) {
TypeInfo type1 = getVariableType(var1);
TypeInfo type2 = getVariableType(var2);
if (type1.type == TYPE_ERROR || type2.type == TYPE_ERROR) {
printf("Error: Undefined variable\n");
return TYPE_ERROR;
}
if (!areTypesCompatible(type1.type, type2.type)) {
printf("Error: Type mismatch in operation %s %c %s\n", var1, op, var2);
return TYPE_ERROR;
}
// Return the more general type
if (type1.type == TYPE_FLOAT || type2.type == TYPE_FLOAT) {
return TYPE_FLOAT;
}
return TYPE_INT;
}
int main() {
printf("Type Checking System\n");
printf("====================\n\n");
addVariable("x", TYPE_INT, 0, 0);
addVariable("y", TYPE_FLOAT, 0, 0);
addVariable("z", TYPE_CHAR, 0, 0);
addVariable("arr", TYPE_INT, 1, 10);
printf("\nType checking operations:\n");
DataType result = checkBinaryOperation("x", "y", '+');
if (result != TYPE_ERROR) {
printf("x + y: Valid operation, result type: %s\n",
result == TYPE_FLOAT ? "FLOAT" : "INT");
}
result = checkBinaryOperation("x", "z", '+');
if (result == TYPE_ERROR) {
printf("x + z: Invalid operation\n");
}
result = checkBinaryOperation("y", "arr", '*');
if (result != TYPE_ERROR) {
printf("y * arr: Valid operation, result type: %s\n",
result == TYPE_FLOAT ? "FLOAT" : "INT");
}
return 0;
}
```

output:

```
kits@kits-Veriton-M200-H610:~/Desktop$ cc typecheck.c
kits@kits-Veriton-M200-H610:~/Desktop$ ./a.out
Type Checking System
==================

Added variable x: int
Added variable y: float
Added variable z: char
Added variable arr: int array

Type checking operations:
x + y: Valid operation, result type: FLOAT
Error: Type mismatch in operation x + z
x + z: Invalid operation
y * arr: Valid operation, result type: FLOAT
```

Implement any one storage allocation strategies(heap, stack, static)

stack.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_STACK_SIZE 100

typedef struct {
    char name[20];
    int size;
    int address;
} StackFrame;

StackFrame stack[MAX_STACK_SIZE];
int top = -1;
int current_address = 1000;  // assume stack starts at address 1000

void push(char* name, int size) {
    if (top >= MAX_STACK_SIZE - 1) {
        printf("Stack overflow! Cannot allocate memory for %s\n", name);
        return;
    }
    top++;
    strcpy(stack[top].name, name);
    stack[top].size = size;
    stack[top].address = current_address;
    current_address += size;
    printf("Allocated %d bytes for '%s' at address %d\n", size, name, stack[top].address);
}

void pop() {
    if (top < 0) {
        printf("Stack underflow! Nothing to pop.\n");
        return;
    }
    printf("Deallocating '%s' of size %d from address %d\n",
           stack[top].name, stack[top].size, stack[top].address);
    current_address -= stack[top].size;
    top--;
}

void displayStack() {
    printf("\nCurrent Stack State:\n");
    if (top < 0) {
        printf("  Stack is empty.\n");
```

```
        return;
    }
    for (int i = top; i >= 0; i--) {
        printf("  %s | size: %d | address: %d\n",
               stack[i].name, stack[i].size, stack[i].address);
    }
    printf("\n");
}

int main() {
    push("A", 4);
    push("Sum", 8);
    push("temp", 4);
    displayStack();

    pop();
    push("result", 4);
    displayStack();

    return 0;
}
```

output:

```
kits@kits-Veriton-M200-H610:~/Desktop$ cc stack.c
kits@kits-Veriton-M200-H610:~/Desktop$ ./a.out
Allocated 4 bytes for 'A' at address 1000
Allocated 8 bytes for 'Sum' at address 1004
Allocated 4 bytes for 'temp' at address 1012

Current Stack State:
  temp | size: 4 | address: 1012
  Sum | size: 8 | address: 1004
  A | size: 4 | address: 1000

Deallocating 'temp' of size 4 from address 1012
Allocated 4 bytes for 'result' at address 1012

Current Stack State:
  result | size: 4 | address: 1012
  Sum | size: 8 | address: 1004
  A | size: 4 | address: 1000
```